

M5cet 3.1

Šifrovaná komunikace peer-to-peer v prohlížeči: zprávy, soubory, hovory a sdílení polohy mezi lidmi, kteří znají společné heslo místnosti. Server zprostředkuje spojení, podrží zprávy pro nepřítomné a nikdy nevidí obsah.

Tato dokumentace popisuje verzi **3.1.0**: signalizační protokol v2, šifrování v3 (Argon2id, slepá ID místností, klíče odesílatelů s forward secrecy, párové klíče, E2EE hovorů), binární přenos a relay souborů, cluster přes Redis, účty s více passkeys a obnovou, role v administraci, neměnný audit, zálohy, metriky a alerty.

Node.js 24

TypeScript 7

React 19

WebRTC DataChannel

AES-GCM 256

ECDSA / ECDH P-256

Argon2id

Ed25519

WebAuthn + PRF

SQLite + SQLCipher

Pro koho je co

Uživatel — režimy, co šifrování chrání. **Vývojář** — architektura, protokol v2, API. **Provozovatel** — nasazení, administrace, audit, bezpečnost.

Co je nového ve 3.1

Šifrování v3 s forward secrecy a slepým ID místnosti, binární přenos, relay souborů a E2EE hovorů, více instancí, více passkeys a obnovovací kód, role, neměnný audit, metriky a alerty. Přehled v seznamu změn 3.1.

Vyhledávání: stiskněte **/** a pište — výsledky se hledají v nadpisech i textu, **↑** **↓** a **Enter** skočí na místo. Tlačítko **PDF** stáhne tutéž dokumentaci k tisku.

Obsah

1. Úvod
2. Režimy a funkce
3. Architektura
4. Signalizační protokol v2
5. Šifrování v2
6. Šifrování v3
7. Soubory, relay a hovory
8. Fronta zpráv pro nepřítomné
9. Účty a passkeys
10. Úložiště
11. Administrace
12. Monitoring a audit
13. API reference
14. Více instancí
15. Nasazení
16. Bezpečnostní opatření serveru
17. Návrhy a roadmapa
18. Změny ve 3.1.0
19. Změny ve 3.0.0

Režimy a funkce

Aplikace běží ve dvou režimech. Oba šifrují stejně; liší se tím, co smí dělat server.

REŽIM	CO DĚLÁ SERVER	CO TO PŘINÁŠÍ
light	Jen signalizace (spojení WebRTC). Nic neukládá.	Nejmenší stopa. Zpráva pro někoho, kdo není online, čeká v lokální frontě prohlížeče.
server-enhanced	Navíc: úložiště, relay pro nepřítomné, Web Push, synchronizace nastavení, proxy souborů, když P2P nejde.	Zprávy dorazí i nepřítomnému (přihlášenému passkey), historie přežije zavření okna, soubory projdou i přes striktní NAT.

Uchování chatu

VOLBA	KDE JE HISTORIE	KDO JI PŘEČTE
ephemeral	Nikde. Nové spojení začíná prázdné.	—
session	V sessionStorage prohlížeče, šifrovaná neexportovatelným klíčem z IndexedDB.	Jen tato záložka prohlížeče.
server s passkey	V trezoru účtu a v jeho SQLCipher databázi; zapečetěno v prohlížeči klíčem z PRF passkey.	Jen držitel passkey (zero-knowledge).
server bez passkey	V dočasné databázi relace (TTL 1 den); každý řádek zapečetěný v prohlížeči klíčem, který zařízení neopustí.	Jen tento prohlížeč. Server drží řádky, které neumí otevřít.

Funkce v kostce

Zprávy

Místnost i soukromí příjemci, odpovědi, přeposílání, stav (odesláno → uloženo → doručeno → přečteno), zprávy *klikací*, *mizející* a *zapečetěné* vlastním kódem.

Soubory

Malé přímo ve zprávě, velké po kusech po DataChannelu; když P2P nejde, přes šifrovanou proxy serveru. Opakování ztracených kusů, ověření otisku celého souboru.

Hovory a média

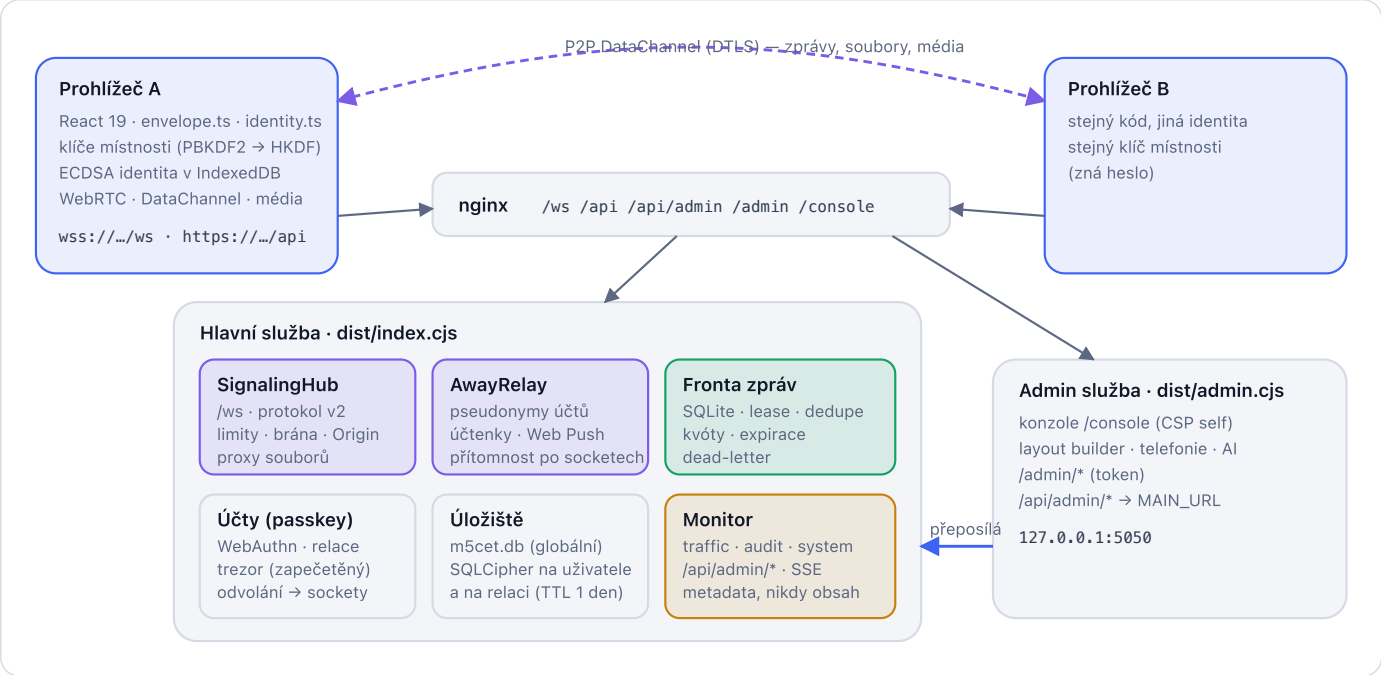
Audio a video přes WebRTC; DTLS otisky chráněné zapečetěnou signalizací a každý rámec navíc šifrovaný klíčem páru (E2EE hovorů). Sdílení polohy, NFC, přepis řeči, AI asistent a telefonie jako volitelné moduly.

Účty

Passkey (WebAuthn) s rozšířením PRF: z něj se v prohlížeči odvodí klíč trezoru i klíč vlastní databáze. Server nezná žádný z nich. Více passkeys a obnovovací kód pro případ ztráty zařízení.

Architektura

Tři běžící části: **prohlížeče** (veškerá kryptografie), **hlavní služba** (Express + WebSocket, všechen živý stav) a **admin služba** (konzole a nástroje). Před nimi obvykle nginx.



Obr. 1 — Části systému a kudy tečou data. Fialové: signalizace a relay; zelené: fronta; oranžové: monitoring.

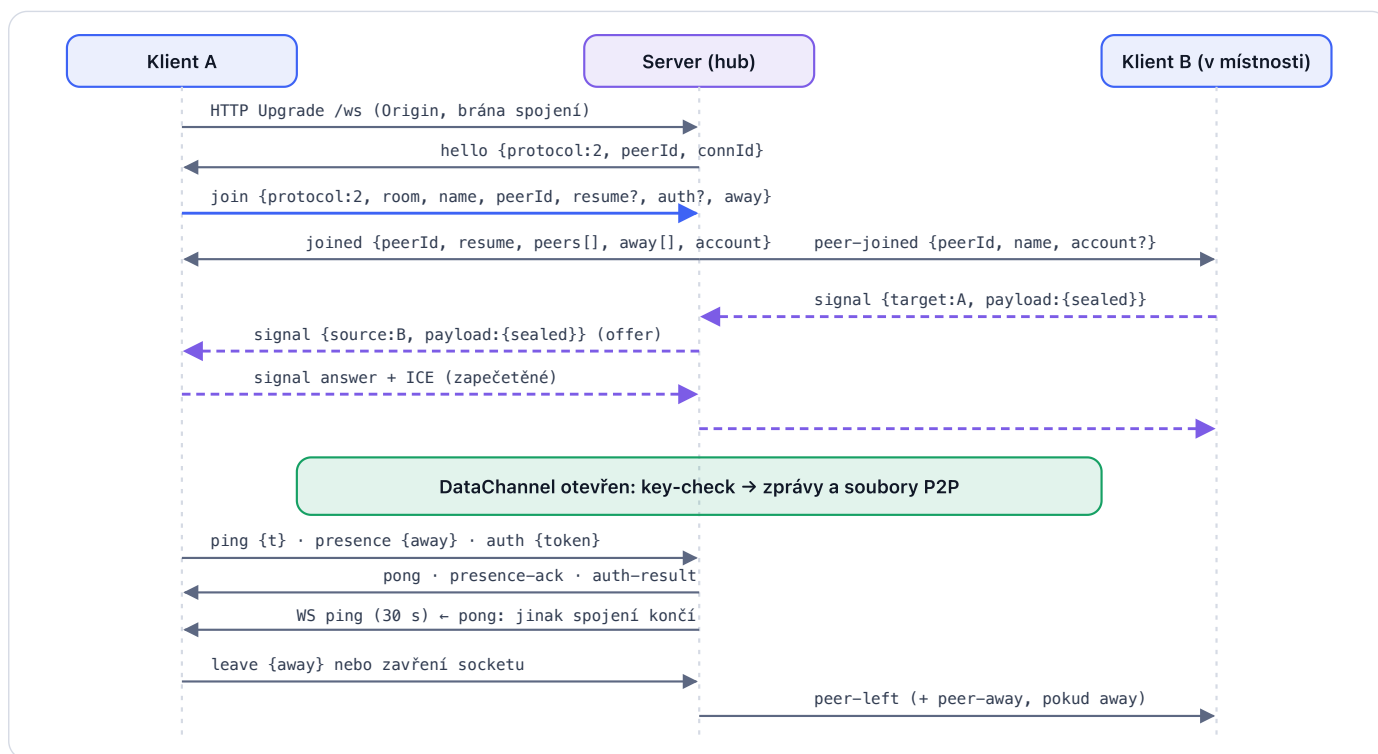
Kde je co v kódu

OBLAST	SOUBORY
Signalizace, protokol v2	server/signaling/hub.ts, frames.ts, limits.ts, refs.ts
Relay pro nepřítomné	server/signaling/relay.ts
Fronta zpráv	server/accounts/mailqueue.ts (SQLite), memqueue.ts (paměť)
Účty, WebAuthn	server/accounts/store.ts, routes.ts, webauthn.ts, cbor.ts
Úložiště	server/storage/* — service.ts, global-store.ts, user-store.ts, keys.ts
Monitoring a audit	server/monitor/traffic.ts, audit.ts, system.ts; API server/admin-api.ts
Admin služba a konzole	server/admin.ts, admin-ui/public/ (console.js, legacy-tools.js)
Kryptografie v prohlížeči	client/src/lib/envelope.ts, identity.ts, crypto.ts (v1), file-transfer.ts, message-kinds.ts
Validace přichozích dat	client/src/lib/validate.ts
Aplikace	client/src/App.tsx a komponenty v client/src/components/

Signalizační protokol v2

Jeden WebSocket na `/ws`, rámce jako JSON objekty s polem `type`. Verze 2 přinesla přesnou validaci, limity, pseudonymy, rámec `auth` a zapečetěnou signalizaci.

Životní cyklus spojení



Obr. 2 — Připojení, výměna zapečetěné signalizace a udržování spojení.

Co se stane s každým rámcem

- Velikost** — nad 256 KB (`MAX_FRAME_BYTES`) knihovna ws spojení zavře kódem `1009`.
- Parsování** (`parseFrame`) do přesného tvaru: typy, délky, formát ID (`[A-Za-z0-9_.-]{1,96}`), base64, počty. Neznámý typ → `error {code:"unknown-type"}`, špatný tvar → `invalid-frame`.
- Limit třídy** — token bucket na socket. Při překročení `rate-limited {frame, retryAfterMs}`; 20 odmítnutí za minutu → zavření `1008`.
- Akce a záznam** do monitoru provozu (typ, bajty, spojení, peer, účet, hash místnosti, stav).

Rámce klient → server

TYPE	POLE	VÝZNAM	TŘÍDA LIMITU
<code>join</code>	<code>protocol</code> , <code>room</code> , <code>name</code> , <code>peerId?</code> , <code>resume?</code> , <code>auth?</code> , <code>away</code>	Vstup do místnosti; <code>auth</code> = session token účtu	signaling
<code>auth</code>	<code>token</code> <code>null</code> , <code>away</code>	Přihlásit / odhlásit účet bez opuštění místnosti	signaling
<code>leave</code>	<code>away</code>	Odchod; <code>away:true</code> = zůstat jako nepřítomný (relay)	signaling
<code>signal</code>	<code>target</code> , <code>payload</code>	SDP/ICE pro jednoho peera; ve v2 <code>{sealed:{v,iv,ciphertext}}</code>	signaling
<code>ping</code>	<code>t</code>	Aplikační heartbeat a měření RTT	heartbeat
<code>presence</code>	<code>away</code>	Stránka odložena / zpět (lifecycle)	presence
<code>relay</code>	<code>messageId</code> , <code>to[]</code> , <code>envelope</code> , <code>expiresAt?</code>	Šifrová obálka pro nepřítomné členy (odkazy z <code>away[]</code>)	relay
<code>relay-ack</code>	<code>ids[]</code> (≤ 500)	Potvrzení zpracování doručených položek	relay
<code>receipt</code>	<code>messageIds[]</code> (≤ 200), <code>state</code>	Přečteno / doručeno — jen pro zprávy, které server tomuto čtenáři doručil	receipt
<code>storage</code>	<code>id</code> , <code>op</code> , <code>payload</code> , <code>auth?</code> , <code>session?</code>	Operace úložiště po WebSocketu (viz úložiště)	storage
<code>command-poll</code> / <code>command-ack</code>	<code>deviceId</code> / <code>commandId</code> , <code>result?</code>	Příkazy operátora pro zařízení	command
<code>proxy-meta</code> <code>chunk</code> <code>end</code> <code>cancel</code> <code>need</code>	<code>transferId</code> , ...	Proxy souborů přes server (když P2P nejde)	proxy (+ bajty)

Rámce server → klient

TYPE	KDY
<code>hello</code>	Hned po otevření: <code>protocol</code> , navržené <code>peerId</code> , <code>connId</code> , <code>limits</code>
<code>joined</code>	Odpověď na <code>join</code> : přidělené <code>peerId</code> , <code>resume</code> , <code>peers[]</code> , <code>away[]</code> , <code>account</code>
<code>peer-joined</code> / <code>peer-left</code> / <code>peer-updated</code>	Změny v místnosti (<code>peer-updated</code> = přihlášení/odhlášení bez odchodu)
<code>peer-away</code> / <code>peer-back</code> / <code>peer-gone</code>	Člen s účtem je nepřítomen / zpět / odhlášen (relay pro něj končí)
<code>signal</code> , <code>signal-undeliverable</code>	Signál od peera / cíl v místnosti není
<code>relay-status</code>	<code>stored</code> <code>forwarded</code> <code>duplicate</code> <code>delivered</code> <code>read</code> <code>rejected</code> pro odeslanou zprávu
<code>relay-deliver</code>	Položky z fronty (po 50) — čekají na <code>relay-ack</code>
<code>presence-ack</code> , <code>auth-result</code> , <code>account-revoked</code>	Stav přítomnosti a účtu na tomto socketu
<code>rate-limited</code> , <code>error</code>	Odmítnutý rámec (<code>code</code> : <code>invalid-frame</code> , <code>unknown-type</code> , <code>not-in-room</code> , <code>proxy-refused</code> , <code>server-error</code>)
<code>replaced</code> , <code>closed-by-server</code>	Stejný klient se připojil znovu (resume) / operátor spojení ukončil
<code>proxy-*</code> , <code>proxy-ack</code>	Přeposlané rámce proxy (s <code>from</code>) a přijetí přenosu
<code>admin-command</code> , <code>storage-result</code> , <code>pong</code>	Příkaz operátora, výsledek operace úložiště, odpověď na ping

Datagramy — rozložení klíčových rámců

☐ hlavička / směrování (vidí server) ☐ bezpečnostní pole ☐ šifrový text (server nepřečte)

join

type	protocol	room	name	
"join"	2	1–64 znaků	≤ 48, bez řídících znaků	
peerId?		resume?	auth?	away
požadované ID		tajemství z minulého joined	session token (≤ 200)	bool

klient → server

joined

type	protocol	peerId	resume
"joined"	2	přidělené — převzít!	32 B base64url, jednorázově
peers[]		away[]	account
{peerId, name, joinedAt, account?}		{account, name, since}	{account, away} {invalid} null

server → klient

signal (zapečetěný)

type	target / source	sealed.v	sealed.iv	sealed.ciphertext
"signal"	peer ID	2	12 B	AES-GCM(klíč „signal“, SDP ICE) · AAD „m5cet/2 signal místnost od pro“

klient → server → klient

relay

type	messageId		to[]	expiresAt?
"relay"	ID zprávy		≤ 50 odkazů účtů (pseudonymy)	TTL zprávy
envelope.v	envelope.id	envelope.iv	envelope.ciphertext	
2	= messageId	12 B	AES-GCM(klíč „message“, {b, pk?, s?}) · AAD „m5cet/2 msg místnost id“ · ≤ 190 KB	

klient → server

relay-deliver — položka

id	seq	kind	messageId	from	
mq-... (pro ack)	pořadí v místnosti	message status		{peerId, name, account?}	
envelope			status?	storedAt	attempts
přesně to, co odesílatel poslal			{state, at, recipientName}		

server → klient

proxy-chunk

type	transferId	seq	v	iv	ciphertext	from
"proxy-chunk"		0 ... n-1	2		AES-GCM(klíč souboru, kus) · AAD „m5cet/2 chunk přenos seq počet“	přidá server

odesílatel → server → místnost

Limity

TŘÍDA	KAPACITA (BURST)	DOPLŇOVÁNÍ / S	RÁMCE
signaling	120	10	join, leave, signal, auth
relay	60	2	relay, relay-ack
receipt	30	1	receipt
presence	10	0,2	presence
storage	120	10	storage
proxy	400	60	proxy-* + bajty: 8 MB burst, 2 MB/s
heartbeat	6	0,5	ping
command	20	0,5	command-poll, command-ack
other	30	1	neplatné rámce

Brána spojení (při upgradu): 30 nových spojení za minutu a 20 současně na adresu, 5 000 celkem. Adresa se bere z `X-Forwarded-For` jen od důvěryhodné proxy (`TRUST_PROXY`). **Zpětný tlak**: příjemci s více než 8 MB ve frontě socketu server přestane posílat kusy souborů (klient si je vyžádá znovu), nad 32 MB je spojení ukončeno.

Peer ID, resume a pseudonymy

- Server v `hello` navrhne ID; klient smí v `join` požádat o vlastní. Je-li v místnosti obsazené jiným živým socketem, dostane nové — **převzetí cizího ID už není možné** (dřív šlo odklonit cizí signalizaci).
- `joined.resume` je jednorázové tajemství. Při novém připojení do stejné místnosti ho klient pošle spolu se svým ID; server porovná SHA-256 v konstantním čase, starý socket dostane `replaced` a ID přejde na nové spojení.
- Členové s účtem se místnosti ukazují jako `account = base64url(HMAC-SHA256(tajemství, místnost || 0 || ID účtu))[0..22]`. Stejná osoba má v každé místnosti jiný odkaz. Pole `accountId` nese totéž (alias pro klienty 2.x).

Příklad: minimální klient

```
const ws = new WebSocket("wss://chat.example.com/ws");
let me = null;
ws.onmessage = (e) => {
  const f = JSON.parse(e.data);
  if (f.type === "hello") ws.send(JSON.stringify({ type: "join", protocol: 2, room: "brno", name: "Alice", peerId: f.peerId, away: false }));
  if (f.type === "joined") { me = f.peerId; /* uložit f.resume pro reconnect */ }
  if (f.type === "rate-limited") setTimeout(retry, f.retryAfterMs);
  if (f.type === "error") console.warn(f.code, f.message);
};
setInterval(() => ws.send(JSON.stringify({ type: "ping", t: Date.now() })), 25_000);
```

Kompatibilita: klient 3.0 přijímá jen zapečetěné signály. Klient 2.x se po nasazení sám nabídne k obnovení; dokud se neobnoví, P2P spojení s ním nevznikne a jeho zprávy v1 klient 3.0 sice přečte, ale 2.x nepřečte zprávy v2.

Šifrování v2

Veškerá kryptografie běží v prohlížeči přes WebCrypto. Server dostává jen šifrované texty a metadata potřebná ke směrování.

Hierarchie klíčů místnosti



Obr. 3 — Z jednoho hesla čtyři nezávislé klíče; žádný z nich neopustí prohlížeč.

Svázání s kontextem (AAD)

Každé šifrování AES-GCM nese *associated data* — řetězec, který se nešifruje, ale je chráněn tagem. Šifrovaný text proto jde otevřít jen tam, kam patří.

ÚČEL	AAD	ČEMU BRÁNÍ
zpráva	<code>m5cet/2 msg <místnost> <id></code>	přenesení do jiné místnosti, záměně ID, přehrání pod jiným ID
signál	<code>m5cet/2 signal <místnost> <od> <pro></code>	přesměrování nebo podvržení SDP serverem (MITM hovorů)
meta souboru	<code>m5cet/2 file-meta <přenos></code>	záměně metadat mezi přenosy
kus souboru	<code>m5cet/2 chunk <přenos> <seq> <počet></code>	přeskládání kusů, vložení kusu z jiného souboru
konec souboru	<code>m5cet/2 file-end <přenos></code>	podvržení otisku
řádek historie na serveru	<code>m5cet:server-row:v1:<id></code>	záměně řádků v úložišti relace

Obálka zprávy

Envelope v2				DataChannel i relay
v	id	iv	ciphertext	
2	ID zprávy (v AAD)	12 B náhodně	AES-GCM-256(JSON { b: tělo, pk?: veřejný klíč, s?: podpis }) + 16 B tag	
tělo b (uvnitř šifrování)				jen příjemci
id	text / attachment	senderId, senderName	createdAt, ttl, flags	to, replyTo, forwardedFrom
= envelope.id	obsah, příloha data:			

Podpis `s` = ECDSA-P256-SHA256 přes `AAD || b` klíčem identity odesílatele; `pk` je jeho veřejný klíč (SPKI). Obojí je **uvnitř** šifrování — server nevidí, kdo co podepsal, a nemůže tak propojit identitu zařízení napříč místnostmi.

```
// client/src/lib/envelope.ts
const keys = await deriveRoomKeys("brno", "correct horse battery");
const me = await loadIdentity(); // ECDSA P-256, IndexedDB
const env = await sealMessage(keys, "msg-1", { id: "msg-1", text: "ahoj", /* ... */ }, me);
// → { v: 2, id: "msg-1", iv: "...", ciphertext: "..." }
const { payload, version, signer } = await openMessage(keys, env);
// signer = { publicKey, valid: true } · version = 2
```

Identita zařízení, TOFU a bezpečnostní čísla

- Každý profil prohlížeče má pár ECDSA P-256; soukromý klíč je **neexportovatelný** (umí podepsat, nejde přečíst ani kódem aplikace) a leží v IndexedDB `m5cet-identity`.
- **Trust on first use**: první ověřený klíč pro jméno v místnosti se připne (`m5cet:pins:v1`). Zpráva pod stejným jménem s jiným klíčem se označí **jiný klíč** a zobrazí se varování; neplatný podpis **neplatný**; starší klient **nepodepsáno**.
- **Bezpečnostní číslo** dvou lidí = 12 skupin po 5 číslicích ze SHA-512 (1 025 kol) seřazených veřejných klíčů — obě strany vidí totéž a mohou ho porovnat mimo aplikaci.
- Otisk vlastního zařízení i ověření odesílatele ukazuje *Informace o zprávě*.

Kontrola klíče a replay

Po otevření DataChannelu si peeři pošlou `{kind:"key-check", v:2, check}`. Nesouhlasí-li, aplikace řekne „jiné heslo místnosti“ místo nečitelných zpráv. Hodnota je odvozená z tajemství místnosti a k útoku na heslo nedává víc než libovolný šifrový text. ID přijatých zpráv si klient pamatuje (20 000 posledních) a opakovaně zahodí.

Soubory

1. Odesílatel vytvoří `transferId`, odvodí **klíč souboru** (HKDF, salt = ID přenosu) a pošle zapečetěná, podepsaná metadata.
2. Každý kus (32 KiB) šifruje s AAD `chunk|přenos|seq|počet` a zároveň počítá jeho SHA-256.
3. Konec přenosu nese zapečetěné a podepsané `{ root, totalChunks, size }`, kde `root` = SHA-256 zřetěžených otisků kusů.
4. Příjemce ověří: metadata před alokací (velikost, počet kusů ≤ 2 000 000, konzistence), každý kus proti AAD, na konci součet bajtů, `root` a shodu podpisu s metadaty. Ztracené kusy si vyžádá (`file-need` / `proxy-need`, max. 3 kola).
5. Jméno souboru se očistí (bez cest a řídicích znaků, BiDi) a **MIME typ se omezí na bezpečné** (obrázky, audio, video, PDF, text). HTML nebo SVG přijde jako `application/octet-stream` — nikdy se nespustí jako stránka v originu aplikace.

Zapečetěné zprávy

Text zprávy se navíc šifruje kódem předaným mimo aplikaci. Ve 3.0: kód **12 znaků** z 31znakové abecedy bez záměnných znaků (≈ 59 bitů, zobrazení `XXXX-XXXX-XXXX`), losování bez modulo biasu, PBKDF2 **600 000** iterací, při zadání se ignoruje velikost písmen, mezery a pomlčky. Zprávy z 2.x (6 znaků, 150 000 iterací) jdou dál otevřít. Text ani kód zapečetěné zprávy se nikdy neukládají mimo prohlížeč.

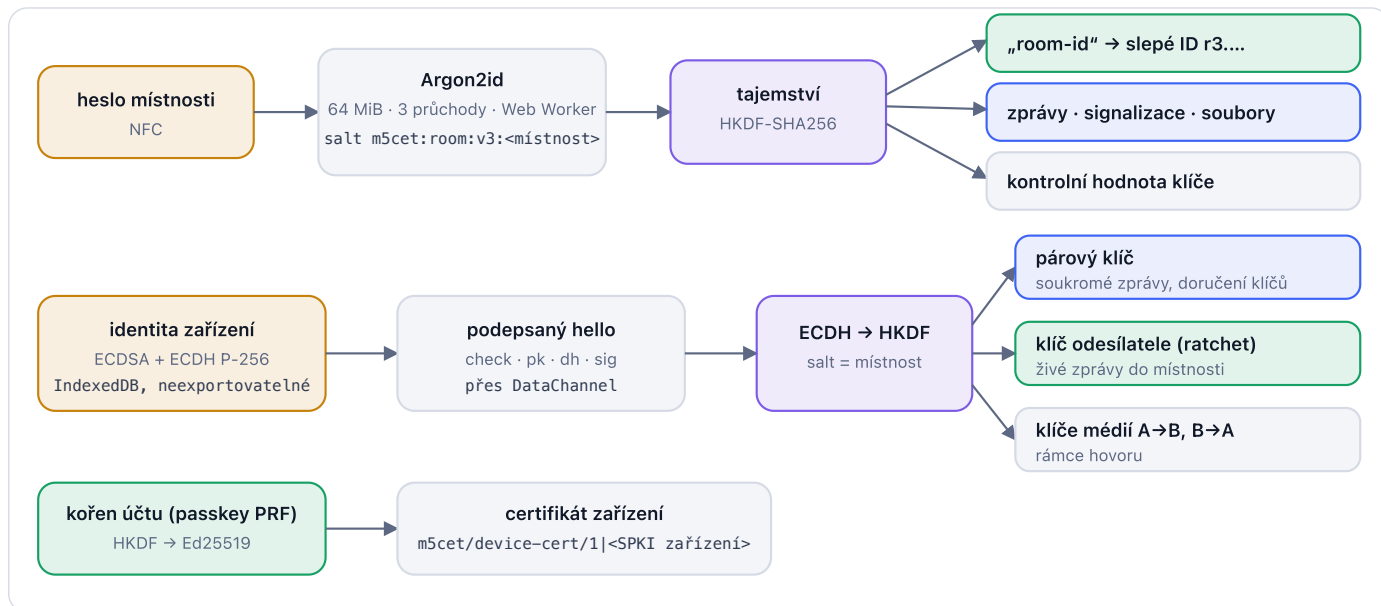
Co kdo vidí

ÚDAJ	SERVER	ČLEN MÍSTNOSTI	POZOROVATEL SÍTĚ
Obsah zpráv, souborů, hovorů	ne	ano (má klíč)	ne
Kdo zprávu napsal (podpis)	ne	ano, ověřeně	ne
SDP / ICE, DTLS otisky	ne (zapečetěno)	ano	ne
Název místnosti	ano (směrování) — od 3.1 ne (slepé ID)	ano	ne (TLS)
Kdo je v místnosti, kdy, IP	ano (monitor ukládá IP zkrácenou)	jména	IP spojení
Kdo komu píše přes relay	ano — do auditu jen se zapnutým auditem komunikace	—	ne
Heslo místnosti, klíče, passkey	nikdy	heslo ano	ne

Ve 3.0 otevřené, ve 3.1 vyřešené: klíč místnosti byl sdílený a statický — kdo heslo zná (nebo ho později získá), přečetl by i starší zachycený provoz. Verze 3.1 šifruje živé zprávy klíči odesílatelů s ratchetem a umí vyloučit člena bez změny hesla — viz Šifrování v3.

Šifrování v3 3.1

Verze 3.1 mění první krok (Argon2id), skrývá před serverem název místnosti a živé zprávy už nešifruje sdíleným klíčem místnosti, ale klíči odesílatelů s ratchetem a párovými klíči. Vše z kapitoly Šifrování v2 (AAD, podpisy, ověřené soubory) platí dál.



Obr. 3b — Klíče místnosti (nahore), klíče párů a odesílatelů (uprostřed), identita účtu (dole).

Argon2id ve Web Workeru

PBKDF2 ztěžuje hádání hesla jen časem; grafická karta ho spočítá paralelně. Argon2id (64 MiB paměti, 3 průchody, `hash-wasm`) stojí útočníka paměť na každý pokus. Výpočet běží v `kdf.worker.ts`, stránka nezamrzne; kde worker nejde spustit, poběží v hlavním vlákně. WebAssembly potřebuje v CSP `'wasm-unsafe-eval'` (nic jiného se nepovoluje). Obálky v2 (PBKDF2, např. zprávy čekající ve frontě z 3.0) se otevrou klíči odvozenými při první potřebě.

Slepé ID místnosti

Klient se připojuje pod `r3.<base64url(HKDF(tajemství, "room-id"), 192 b)>`. Server tak směřuje podle hodnoty, ze které název nezjistí; lidé se stejným názvem a jiným heslem se ani nepotkají. Historie na serveru se čte pod slepým ID, se zpětnou cestou pro data uložená 3.0 pod názvem.

Hello a párové klíče

hello DataChannel, po otevření						
kind	v	check	pk	dh	sig	caps
"hello"	3	kontrolní hodnota	ECDSA klíč zařízení (SPKI)	ECDH P-256 klíč	podpis m5cet/hello/1 místnost od pro check dh	bin, media

Podpis váže DH klíč na klíč zařízení, který si protistrany pinují. Jiná kontrolní hodnota = jiné heslo (aplikace to řekne), neplatný podpis = varování. Párový klíč = `HKDF(ECDH, salt = místnost, info = "m5cet/pair/1" + oba klíče zařízení seřazené)` — znají ho právě tato dvě zařízení. Soukromá zpráva vybraným příjemcům se zapečetí pro každého zvlášť (`sk: "pair"`); jiný člen místnosti ji neotevře ani s heslem.

Klíče odesílatele a forward secrecy

Každý odesílatel drží řetězový klíč `CK` a po každé zprávě ho posune:

```

MKn  = HMAC(CKn, 0x01)    // klíč zprávy n (AES-GCM), po použití se zahodí
CKn+1 = HMAC(CKn, 0x02)    // další článek; starý se přepíše
obálka = { v: 3, id, sk: <ID řetězu>, n, iv, ciphertext }
```

- Aktuální `CK` dostane každý peer zapečetěný **párovým klíčem** (`{kind:"sender-key", iv, ct}`) — server ho nevidí, jiný člen ho nepřečte.
- Kdo později získá heslo, **starší zachycené zprávy nepřečte**: klíče už přečtených zpráv nejdou z ničeho v paměti znovu spočítat.
- Řetěz se vymění po **500 zprávách, hodině**, nebo když někdo **odejde**; noví dostanou jen ti, kdo zůstali. Mimo pořadí se přijme až 1 000 přeskočených zpráv.

- **Vyloučit z mých zpráv** (Informace o účastníkovi): spojení skončí, klíč zařízení se na relaci zablokuje a řetěz se vymění.
- Klíčem místnosti se dál šifrují zprávy pro **nepřítomné** (relay), fronta odchozích zpráv, stav mikrofonu, signalizace a soubory — tam příjemce párový kanál mít nemusí.

Identita účtu a bezpečnostní čísla

- Z kořene účtu (PRF passkey) se odvodí podpisový klíč **Ed25519** (`HKDF info "m5cet:account-sign:v1"`). Podepíše klíč zařízení (`m5cet/device-cert/1|<SPKI>`); podepsané zprávy nesou klíč účtu i certifikát.
- Pin se od 3.1 váže na **účet**, ne na jméno: nové zařízení téhož účtu nevyvolá varování, cizí klíč pod stejným jménem ano. Odznak zprávy: **ověřený účet** , **ověřeno bezpečnostním číslem** .
- **Bezpečnostní číslo** (60 číslic) se dá porovnat i jako **QR kód** (`M5CET-SN:1:<číslice>`) — zobrazit, naskenovat kamerou druhého (`BarcodeDetector`), potvrdit „ověřeno“.
- Každá zpráva v detailu říká, čím byla zapečetěná: klíčem odesílatele, párovým klíčem, nebo klíčem místnosti.

Co kdo vidí ve 3.1

ÚDAJ	3.0	3.1
Název místnosti pro server	ano	ne — slepé ID
Starší provoz po úniku hesla	čitelný	nečitelný (živé zprávy; relay a fronta dál pod klíčem místnosti)
Soukromá zpráva pro ostatní členy	čitelná s heslem	nečitelná — párový klíč
Rámce hovoru pro prvek, který ukončí SRTP (SFU, relay)	čitelné	nečitelné — E2EE rámců
Kdo komu psal (fronta v DB)	čitelně v globální DB	zapečetěné master klíčem, odesílatel jen jako hash
Text probuzení na zamčené obrazovce	jméno, místnost	neutrální

Soubory, relay a hovory 3.1

Binární rámce

Kus souboru už necestuje jako JSON s base64 (o třetinu víc dat a práce na obou stranách), ale jako jedna binární zpráva — po DataChannelu i po signalizačním socketu:

binary chunk client/src/lib/binary-frames.ts · server/signaling/binary.ts							
0x4D „M“	type 0x01 P2P · 0x11 relay	version	L délka ID	transferId L bajtů	seq u32 BE	IV 12 B	ciphertext + tag AES-GCM, AAD chunk přenos seq počet

Schopnost si strany sdělit: peer v `hello` (`caps: ["bin"]`), klient v `join` (`features: ["bin"]`), server v úvodním `hello` (`features` a `limits.proxy`). Starší klient dostane tentýž kus jako JSON — server ho převede, ale neotevře.

Relay souborů, když P2P nejde

Za striktním NAT bez TURN se DataChannel neotevře. Aplikace pak pošle soubor šifrovaně přes server (`proxy` transport) — dřív přenos rovnou odmítla. Server kusy jen předává (žádná kopie v paměti), hlídá vlastnictví přenosu a strop 64 souběžných přenosů. Odesílatel se řídí rozpočtem, který server ohlásí (2 MiB/s, 60 rámců/s na socket, s rezervou): posílá na **80 %**, polovinu burstu, a čeká, dokud vlastní buffer socketu neklesne pod 1 MiB. Relay tak kus nikdy neodmítne ani spojení neodpojí.

E2EE hovorů

Hovor běží DTLS-SRTP mezi prohlížeči a signalizace, která ho sestavuje, je zapečetěná. Navíc se **každý zakódovaný rámec** zvuku i obrazu zapečetí v workeru (`RTCTranscriptTransform`) klíčem páru pro daný směr — `HKDF(ECDH, info "m5cet/media/1|odesílatel|příjemce")`. Rámec projde i prvem, který SRTP ukončí (SFU, záznamový relay), a zůstane nečitelný.

rámec hovoru client/src/lib/media-frames.ts				
čitelný prefix Opus 1 B · VP8 10 / 3 B (klíčový / delta)	AES-GCM ciphertext + tag AAD = prefix	IV sůl 4 B + čítač 8 B	n délka prefixu	0x6D 0xE3 značka

- Transformace odesílání i příjmu se nastaví **na celý transceiver najednou** (při přidání stopy nebo příchodu cizí) — Chrome pozdější transformaci druhého směru tiše ignoruje.
- Bez klíče rámec projde beze změny (hovor chrání DTLS-SRTP); rámec, který nejde otevřít, se zahodí. Panel *Hlas* ukazuje stav za poslední sekundu: **E2EE** jen když byl každý rámec oběma směry zapečetěný a otevřený.
- Do hovoru se může přidat kterákoli strana: vyjednávání je „perfect negotiation“ (kolizi nabídek vyhraje iniciátor spojení).

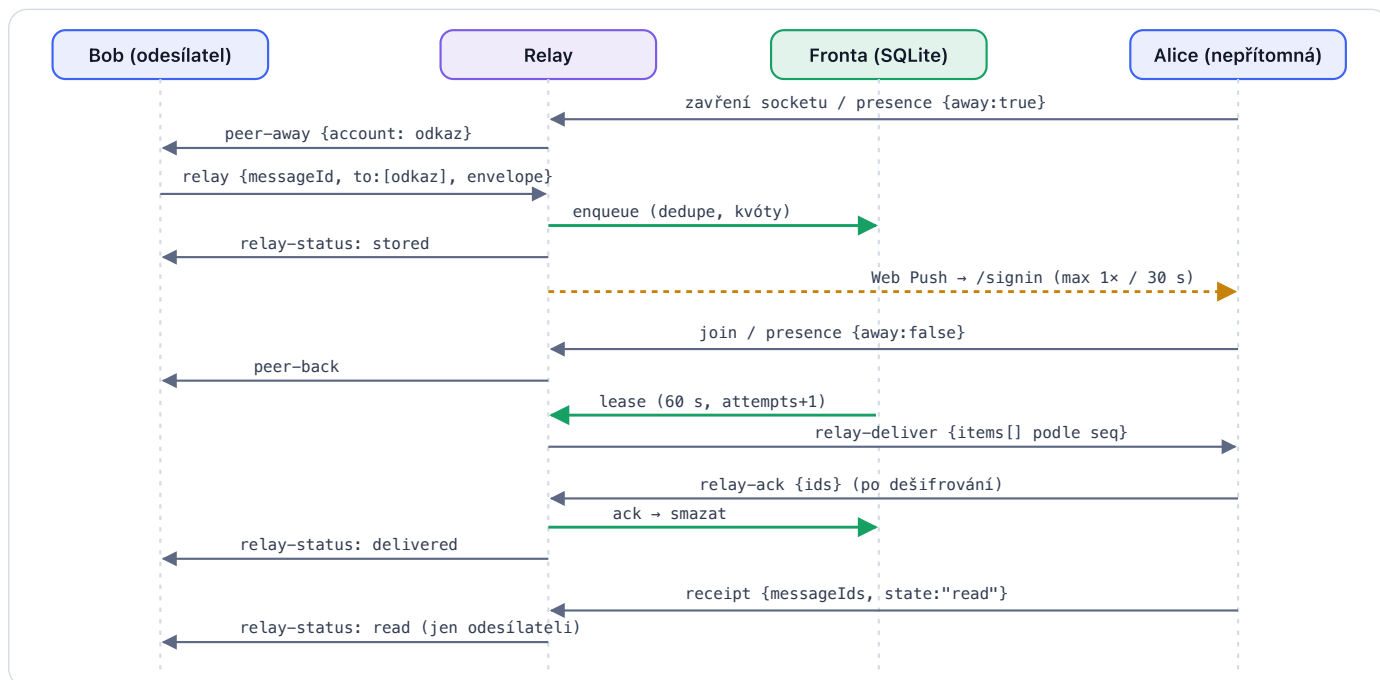
TURN s krátkodobými přístupy

S `TURN_SECRET` (coturn `use-auth-secret`) vydá `/api/turn` jméno `<expirace>:<náhoda>` a heslo `base64(HMAC-SHA1(secret, jméno))`, platné `TURN_TTL_SECONDS` (výchozí hodina). Klient si je obnoví 5 minut před koncem. Statické `TURN_USERNAME` / `TURN_CREDENTIAL` dál fungují, ale server při startu varuje — jsou čitelné pro každého návštěvníka.

Fronta zpráv pro nepřítomné

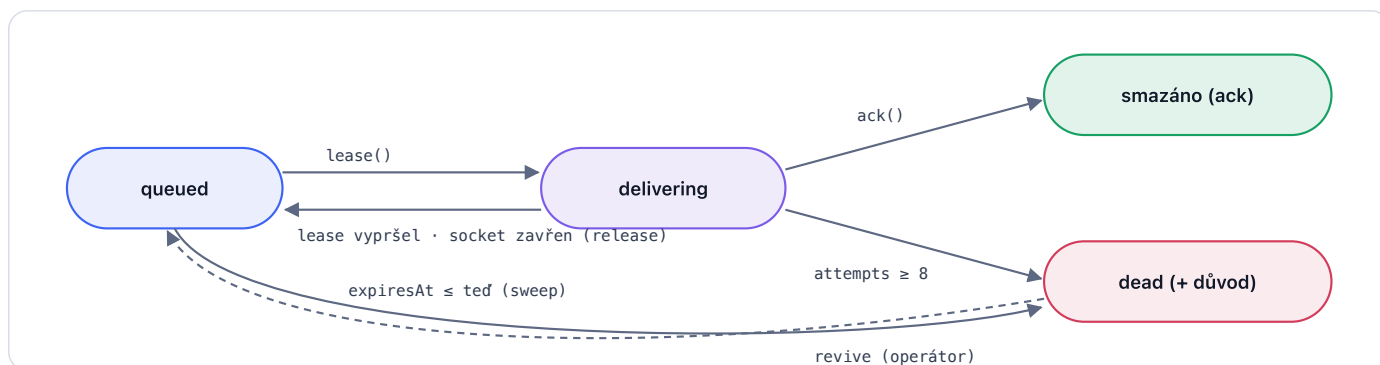
Když člen s účtem (passkey) a zapnutým uchováním na serveru odejde nebo odloží stránku, zůstane v místnosti jako *nepřítomný*. Ostatní mu dál píší; server šifrovou obálku uloží, odpoví „uloženo“, probudí ho Web Push a po návratu mu vše předá — v pořadí a s potvrzením.

Tok relay



Obr. 4 — Od odeslání nepřítomné po „přečteno“. Je-li odesílatel mezitím pryč, stav čeká v jeho vlastní frontě jako položka `status`.

Stavy položky



Obr. 5 — Doručení položku nemaže; smaže ji až potvrzení příjemce.

VLASTNOST	HODNOTA	POZNÁMKA
Pořadí	<code>seq</code> na (účet, místnost)	doručení v pořadí odeslání
Deduplikace	(účet, druh, messageId)	opakovaný relay = <code>duplicate</code>
Položek na účet	1 000	<code>maxItemsPerAccount</code>
Bajtů na účet	8 MB	<code>maxBytesPerAccount</code>
Velikost položky	130 000 B	<code>maxItemBytes</code>
Od jednoho odesílatele	300 položek	jeden člen nezaplní cizí schránku
Lease	60 s	po zavření socketu uvolněn hned
Pokusů	8	pak <code>dead</code>
Život	30 dní (<code>RELAY_RETENTION_DAYS</code>) nebo TTL zprávy	kratší z obou
Dead-letter	7 dní	pak se maže

Poctivé účtenky

Server si ke každé relayované zprávě pamatuje (v paměti, až 50 000 zpráv / 30 dní) odesílatele a příjemce. `receipt` přijme jen od účtu, kterému zprávu doručil, a pošle ho jen jejímu odesílateli. Ve 2.x mohl kdokoli v místnosti označit cizí zprávu jako přečtenou a vložit položky do cizí schránky.

Lokální fronta (light režim)

Bez serveru čeká zpráva, pro kterou není otevřený kanál, v paměti prohlížeče — už zašifrovaná. Zkouší se při otevření kanálu, po návratu do okna a jednou za minutu (200 zpráv, 60 pokusů, 24 h, respektuje TTL). Protože peer ID je od 3.0 stabilní i po výpadku (resume), obálky ve frontě zůstávají platné.

Účty a passkeys

Účet je passkey (WebAuthn) — žádné heslo. Server uloží jen veřejné klíče a počítaadla podpisů; bez rozšíření **PRF** účet nevznikne, protože z něj prohlížeč odvozuje **kořen účtu** a z něj klíče:

- `HKDF(kořen, "m5cet:profile:v1")` → klíč trezoru (profil + historie chatu, zapečetěné před odesláním),
- `HKDF(kořen, "m5cet:userdb:v1")` → klíč vlastní SQLCipher databáze; server ho drží jen v paměti, dokud je uživatel přihlášený,
- `HKDF(kořen, "m5cet:account-sign:v1")` → podpisový klíč účtu Ed25519 (identita účtu).

Více passkeys a obnovovací kód 3.1

Kořenem je PRF prvního passkey. Pro každý další passkey (max. 10) i pro obnovovací kód uloží server kořen **zapečetěný** (AES-GCM, AAD `m5cet:account-root:v1`, klíč `HKDF(PRF nebo tajemství kódu, "m5cet:root-wrap:passkey:v1" | "...recovery:v1")`) — přečíst ho umí jen ten, kdo passkey nebo kód drží. Přidání passkey i vytvoření kódu nejdřív potvrdí stávající passkey.

MECHANISMUS	CHOVÁNÍ
Obnovovací kód	26 znaků Crockford base32 (130 bitů), zobrazí se jednou; z kódu se odvodí ID, důkaz a tajemství (HMAC). Server zná jen ID a <code>SHA-256(důkaz)</code> . Obnova: <code>recovery/start</code> (10 pokusů za hodinu z adresy) vydá lístek na 10 minut a zapečetěný kořen, prohlížeč ho otevře, zaregistruje nový passkey a <code>recovery/finish</code> ho přidá k účtu.
Relace	náhodný token (32 B), na serveru jen SHA-256 v <code>sessions.json</code> — přežijí restart ; klouzavá platnost 12 h od posledního použití, nejdéle 7 dní, max. 20 na účet
Zařízení	okno účtu ukazuje relace (klient, adresa /24, poslední použití) a každou jde ukončit; aktuální je označená
Výzvy	náhodné, jednorázové, 2 minuty, vázané na obřad (registrace, přihlášení, přidání passkey, obnova)
Odvolání	odhlášení (jedno zařízení), „všude“, smazání účtu, zásah operátora → <code>account-revoked</code> na všech dotčených socketech (i na jiných instancích), relay pro účet končí, databáze se pro toto zařízení zamkne
RP ID	<code>WEBAUTHN_RP_ID</code> , jinak host z <code>PUBLIC_BASE_URL</code> , jinak hlavička Host (v produkci server varuje)
Push	jen endpointy známých služeb (FCM, Mozilla, Windows, Apple; rozšíření <code>PUSH_ENDPOINT_HOSTS</code>); text probuzení je neutrální

Úložiště

Jedna globální SQLite databáze pro server a jedna SQLCipher databáze pro každého uživatele či anonymní relaci. Podrobně v `docs/storage.md`.

```
$DATA_DIR/storage/
├─ m5cet.db      # globální: uživatelé, passkeys, index DB, logy, přenosy, audit, fronta
├─ storage.key   # master klíč, 0600 (nebo STORAGE_MASTER_KEY / STORAGE_KEY_FILE)
└─ db/
   ├─ db-<id>.db # účet – klíč z passkey (PRF), jen v paměti
   └─ db-<id>.db # relace – klíč zabalený master klíčem, TTL 24 h od posledního použití, max. 7 dní
```

Limity

Kvóta 64 MB na účet, 16 MB na relaci (zápis nad kvótou se vrátí); zpráva ≤ 256 KB; hodnota kv ≤ 4 MB; 30 nových relací za hodinu z jedné adresy, 5 000 živých celkem; nejvýš 200 otevřených databází (LRU).

Povýšení relace na účet

Po vytvoření passkey se data relace zkopírují do databáze účtu v jedné transakci (ATTACH s klíčem), počty se ověří před commitem; relace se smaže až potom.

Klíče

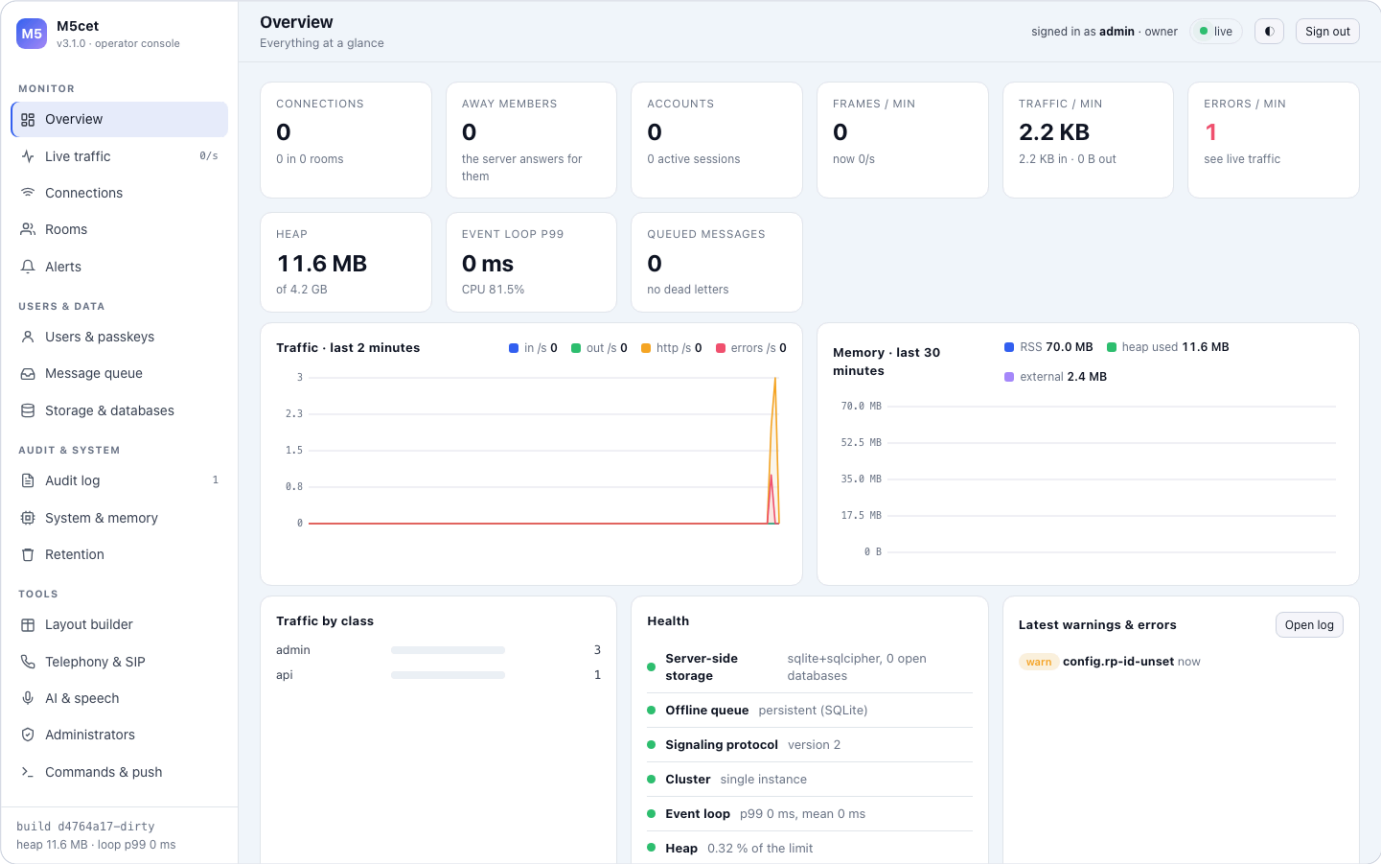
Klíč účtu drží server po držitelích (zařízeních); zapomene ho a přepíše nulami, až se odhlásí poslední, nebo po 12 h nečinnosti. ID relací jen jako HMAC.

Veřejný status

`/api/storage/status` říká jen `available`, `engine`, `caller`, `locked` — nic o ostatních uživateli. Úplný stav je v administraci.

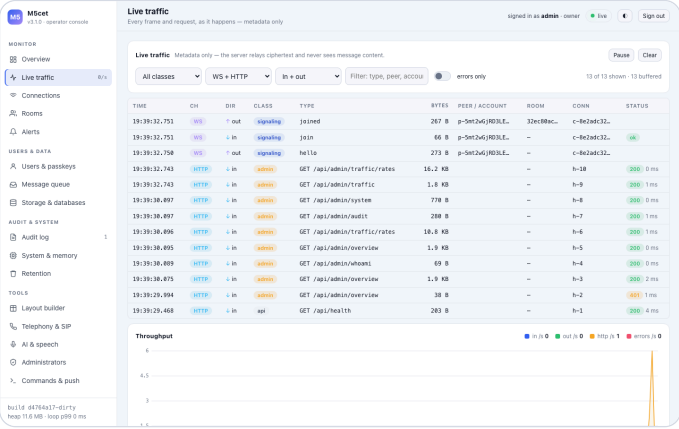
Administrace

Konzole operátora na `https://<doména>/console/`. Živá data bere z hlavní služby (`/api/admin/*`), nástroje layoutu, telefonie a AI z admin služby (`/admin/*`). Přístup mají jmenovití administrátoři s rolí; `ADMIN_API_TOKEN` zůstává jako vlastník „admin“.

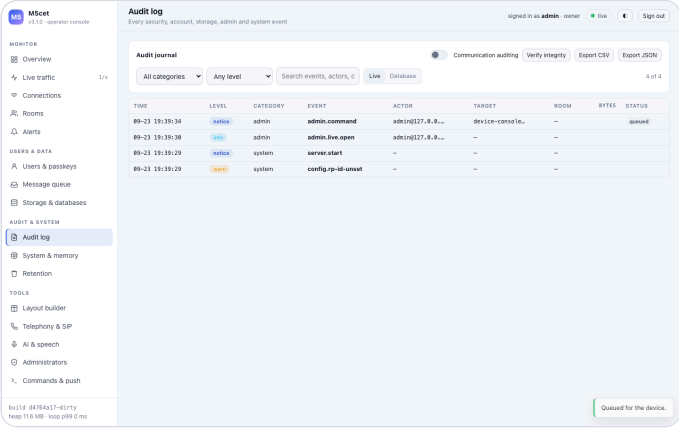


Obr. 6 — Přehled: KPI, provoz za 2 minuty, paměť za 30 minut, provoz podle tříd, zdraví, poslední varování.

PANEL	CO UKAZUJE A CO UMÍ
Přehled	spojení, místnosti, nepřítomní, účty a relace, rámce a bajty za minutu, chyby, heap, event loop p99, fronta; grafy; zdravotní kontroly (úložiště, trvalost fronty, event loop, heap, audit komunikace)
Živý provoz	každý WebSocket rámec a HTTP požadavek (čas, kanál, směr, třída, typ, bajty, peer/účet, hash místnosti, spojení, stav, doba); filtry, pauza, detail; graf propustnosti
Spojení	otevřené sockety: jméno, peer, účet, místnost, klient (prohlížeč/OS), adresa /24 (/48), bajty a rámce, doba, protokol; odpojení
Místnosti	členové online i nepřítomní (hash místnosti, nikdy název)
Uživatelé a passkeys	účty, databáze (klíčování, otevřená), trezor, fronta, relace; detail s passkeys, položkami ve frontě, auditem účtu, žurnálem a provozem; odhlásit všude, smazat (s opsáním ID)
Fronta zpráv	stav fronty (uloženo / doručuje se / mrtvé / velikost / trvalost), schránky po účtech, dead-letter s obnovou
Úložiště	globální DB (soubor, WAL, stránky, žurnál, tabulky a řádky), index šifrovaných databází
Audit	žurnál s filtry (kategorie, úroveň, text), zdroj paměť / databáze, export CSV a JSON, přepínač auditu komunikace
Systém a paměť	RSS, heap (a limit), external, event loop p99 / průměr, CPU, zátěž hostitele, otevřené zdroje; grafy 30 minut
Retence	politika uchovávání, další běh, ruční spuštění
Příkazy a push	příkaz pro zařízení (doručí se hned, je-li připojené), čekající příkazy, log doručení, stav VAPID, odběratelé, testovací push
Layout, Telefonie, AI	přenesené nástroje z předchozí administrace



Obr. 7 — Živý provoz.



Obr. 8 — Auditní žurnál.

Role a přístupy 3.1

ROLE	SMÍ
auditor	číst všechno (přehled, provoz, spojení, uživatele, frontu, úložiště, audit a jeho ověření, metriky, alerty); spravovat vlastní passkey. Tlačítka, která něco mění, má zašedlá — a server je odmítne (403).
operátor	navíc zasahovat: odpojit spojení, odhlásit nebo smazat účet, obnovit položku fronty, příkazy a testovací push, retence, zálohy a kontrola integrity, prahy alertů, kontrolní bod auditu
vlastník	navíc spravovat administrátory a jejich tokeny, VACUUM

- **Odkud se berou:** `ADMIN_API_TOKEN` (vlastník), `ADMIN_TOKENS="alice:operator:token,bob:auditor:token"` v prostředí, a administrátoři založení v konzoli (panel *Administrátoři*) s pojmenovanými tokeny — v `admin-users.json` jen jejich hashe.
- **Přihlášení passkey:** administrátor si v konzoli zaregistruje passkey (*Můj passkey*) a přistě se přihlásí bez tokenu; relace konzole platí 8 h.
- Každý zásah se v auditu запиše **jménem** administrátora (`actor`), ne „admin“.

Bezpečnost konzole

- Token je jen v paměti záložky (volitelně v `sessionStorage`), **nikdy v `localStorage`** — starý klíč `m5cet:admin:cfg` konzole při načtení smaže.
- CSP `script-src 'self'`, žádné inline skripty ani `eval`, `frame-ancestors 'none'`. Vše ze serveru jde do stránky přes `textContent`.
- Živý stream je Server-Sent Events čtený přes `fetch()` — token v hlavičce, ne v URL (nezůstane v logu proxy).
- Odmítnuté tokeny se počítají: 30 za 15 minut na adresu. Každý zásah operátora se запиše do auditu (kategorie `admin`).
- Doporučeno omezit `/console/`, `/admin/` a `/api/admin/` v nginx na adresy správců (`allow ... ; deny all;`).

Monitoring a audit

Záznam provozu

Monitor drží v paměti posledních 5 000 záznamů, počítadla po třídách, sekundové řady za 10 minut a registr živých spojení. Obsah zpráv nemá — server ho nikdy nevidí.

TrafficRecord							SSE event „traffic“	
id	at ms	channel ws http	direction in out	cls třída	type „join“, „GET /api/...“		bytes	
conn	peerId	accountId	roomHash SHA-256, 16 hex	ip /24 · /48	status ok · error · dropped · HTTP		durationMs	

Třídy: `signaling`, `presence`, `relay`, `storage`, `file-proxy`, `heartbeat`, `account`, `admin`, `push`, `api`, `static`, `error`, `other`. Heartbeaty se do živého streamu neposílají (jejich počet je v ticku).

Auditní žurnál

KATEGORIE	PŘÍKLADY UDÁLOSTÍ	ZAPNUTO
<code>security</code>	<code>ws.origin-refused</code> , <code>ws.kicked</code> , <code>auth.invalid-token</code> , <code>join.peer-id-taken</code> , <code>relay.refused</code> , <code>http.rate-limited</code>	vždy
<code>account</code>	<code>account.register</code> , <code>account.sign-in</code> , <code>account.deleted</code> , <code>session.revoked</code> , <code>relay.away</code> , <code>relay.back</code> , <code>relay.push</code>	vždy
<code>communication</code>	<code>room.join</code> , <code>room.leave</code> , <code>relay.stored</code> , <code>relay.delivered</code>	jen na vyžádání (<code>AUDIT_COMMUNICATION=1</code> nebo přepínač)
<code>storage</code>	<code>queue.sweep</code> , zakládání, povýšení a mazání databází	vždy
<code>admin</code>	<code>admin.command</code> , <code>admin.user.delete</code> , <code>admin.audit.export</code> , <code>admin.connection.close</code>	vždy
<code>network</code>	<code>ws.refused</code> , <code>ws.closed</code> , <code>ws.heartbeat-timeout</code> , <code>ws.slow-consumer</code>	vždy
<code>system</code>	<code>server.start</code> , <code>server.stop</code> , <code>http.error</code> , <code>config.rp-id-unset</code> , <code>queue.migrated</code>	vždy

Úrovně `debug < info < notice < warn < error`. Žurnál drží 3 000 posledních záznamů v paměti; s úložištěm jde každý záznam i do tabulky `audit` globální databáze, kde je volné pole `detail` **zapečetěné** master klíčem a svázané s kategorií, událostí a časem.

Audit komunikace zaznamenává, kdo koho kdy a jak velkou zprávou oslovil — tedy přesně metadata, která by šifrovaný chat neměl sbírat. Proto je vypnutý a jeho zapnutí (i vypnutí) se samo zapíše do auditu. Zapínejte jen s právním důvodem.

Živý stream

```
GET /api/admin/live?streams=traffic,audit,tick
Authorization: Bearer <ADMIN_API_TOKEN>
Accept: text/event-stream

event: traffic
data: {"id":812,"at":1790000000000,"channel":"ws","direction":"in","cls":"relay","type":"relay","bytes":1843,"conn":"c-1a82ee...","status":"ok"}

event: audit
data: {"id":77,"category":"security","level":"warn","event":"ws.kicked","status":"rate limits exceeded"}

event: tick    (každé 2 s)
data: {"counts":{"..."},"rate":{"framesIn":12,"framesOut":15,"http":1,...},"system":{"rss":..., "loopP99":1.4}}
```

Konzoli, která nestíhá (víc než 1 MB ve frontě odpovědi), server záznamy zahazuje — nikdy kvůli ní nehromadí paměť.

Neměnný audit 3.1

Každý záznam v databázi nese `hash = SHA-256(kanonický JSON [hash předchozího, čas, kategorie, úroveň, událost, ..., detail])`. Každých 500 záznamů, před promazáním starých a na požádání se hlava řetězu **podepíše** (Ed25519, `m5cet/audit-checkpoint/1|poslední_id|hash|čas`; klíč `audit-signing.key` vedle master klíče). Ověření (konzole > Audit > *Ověřit integritu*, `GET /api/admin/audit/verify`) přepočítá řetěz a řekne, kde byl záznam změněn, odstraněn nebo kontrolní bod přepodepsán. Záznamy z doby před 3.1 hlásí jako „před řetězem“.

Metriky a alerty 3.1

`GET /metrics` (token `METRICS_TOKEN` nebo administrátor) vrací Prometheus text: spojení, místnosti, členy, účty a relace, rámce a bajty po třídách, frontu, heap, event loop, CPU, audit, zálohy, cluster (`m5cet_cluster_*`) a počet spuštěných alertů.

PRAVIDLO	VÝCHOZÍ PRÁH	PROMĚNNÁ
bezpečnostní varování za minutu	20	<code>ALERT_SECURITY_WARNINGS</code>
chyby za minutu	120	<code>ALERT_HTTP_ERRORS</code>
event loop p99	200 ms	<code>ALERT_EVENT_LOOP</code>
heap z limitu	85 %	<code>ALERT_HEAP</code>
mrtvé položky fronty	1	<code>ALERT_DEAD_LETTERS</code>
selhání zálohy, integrity nebo řetězu auditu	1	<code>ALERT_STORAGE</code>

Alert se spustí a vyřeší jednou (žádné opakování každou minutu); s `ALERT_WEBHOOK_URL` ho server pošle jako JSON (Slack/Teams/vlastní přijímač). Prahy jde měnit v konzoli (panel *Alerty*, uloží se do `alert-rules.json`).

Zálohy a integrita 3.1

Záloha je adresář `$BACKUP_DIR/m5cet-<čas>/`: globální databáze zkopírovaná online backup API SQLite (za běhu konzistentní), kopie šifrovaných databází uživatelů (zůstávají šifrované), index účtů se zapečetěnými trezory, administrátoři (jen hashe tokenů) a `manifest.json` s velikostí a SHA-256 každého souboru. **Master klíč do zálohy nepatří** — uložte ho zvlášť. S `BACKUP_DIR` běží zálohy samy (`BACKUP_INTERVAL_HOURS`, výchozí 24; drží se `BACKUP_KEEP`, výchozí 7) a s nimi `PRAGMA quick_check`; jinak na požádání z konzole (Úložiště). Tamtéž optimalizace a `VACUUM` (jen vlastník).

API reference

Veřejné REST API hlavní služby

METODA A CESTA	POPIS	AUTORIZACE
GET /api/health	živost, verze, build, protocol, počet místností	—
GET /api/modules	manifest volitelných modulů	—
GET /api/turn	ICE servery; s TURN_SECRET krátkodobé přístupy a expiresAt	—
GET /metrics 3.1	Prometheus metriky	METRICS_TOKEN nebo administrátor
GET /api/layout	vzhled a šablony od operátora	—
POST /api/share/create redeem revoke	pozvánky (split-key, kód 12 číslic); limit 30 / 40 za 15 min	—
GET /api/push/status, POST /api/push/subscribe test	Web Push pro zařízení bez účtu (jen známé push služby)	test sobě bez tokenu, všem s admin tokenem
/api/account/*	registrace a přihlášení passkey, me, trezor, push, odhlášení, smazání	Bearer token relace
POST /api/account/passkeys/options verify, GET passkeys/:id/wrapped, DELETE passkeys/:id 3.1	další passkey účtu, jeho zapečetěný kořen, odebrání (poslední nejde)	token relace
PUT DELETE /api/account/recovery, POST recovery/start finish 3.1	obnovovací kód: nastavit, zrušit, obnovit účet (10 pokusů / h)	token relace / důkaz z kódu
GET /api/account/sessions, DELETE sessions/:id, PUT identity 3.1	zařízení účtu a jejich odhlášení; veřejný klíč účtu (Ed25519)	token relace
/api/storage/*	úložiště (i přes WebSocket rámec storage)	relace účtu nebo X-M5cet-Session
GET POST /api/settings, POST /api/audit/purge, /api/analytics/consent	nastavení a souhlasy vázané na deviceId	znalost deviceId
/api/ai/*, /api/speech/*, /api/telephony/*	volitelné moduly (ENABLE_AI, ENABLE_SPEECH, ENABLE_TELEPHONY)	limity na IP
GET /api/events/recent, GET /api/transfers/stats	metadata událostí, statistika proxy	admin token (od 3.0)

Operátorské API (/api/admin/*)

Všechny cesty vyžadují Authorization: Bearer <token administrátora> (nebo relaci konzole po přihlášení passkey). Čtení (GET) stačí auditorovi, změny operátorovi, správa administrátorů vlastníkoví — viz Role.

METODA A CESTA	POPIS
GET overview	verze, proces, počty, provoz, fronta, úložiště, audit, zdraví
GET live	SSE: traffic, audit, tick (výběr ?streams=)
GET traffic	záznamy provozu; filtry cls, channel, direction, conn, peer, account, room, type, since, before, errors=1, limit
GET traffic/rates?seconds=	sekundové řady (max 600 s)
GET connections, POST connections/:id/close	živá spojení, odpojení
GET rooms	místnosti (hash), členové, nepřítomní
GET users, GET users/:id	účty; detail s passkeys, frontou, auditem, žurnálem, provozem
POST users/:id/signout	odvolat všechny relace (okamžitě i na socketech)
DELETE users/:id?confirm=:id	smazat účet se vším (ID se musí zopakovat)
GET queue, GET queue/dead, POST queue/:id/revive	fronta po účtech, dead-letter, obnova položky
GET system, GET db	proces a paměť (+ historie 30 min); databáze
GET audit (?source=db), GET audit/export?format=csv json	žurnál z paměti nebo databáze; export (CSV s ochranou proti vzorcům)
PUT audit/settings	{ communication: bool }
GET POST commands	povolené příkazy, čekající, log; zařazení příkazu {kind, deviceId, payload?}
GET push, POST push/test	stav VAPID a odběratele; testovací push
GET events	metadata událostí (LOG_EVENTS=1)
GET retention, POST retention/run	politika uchovávání, ruční běh
GET storage, GET storage/logs	index a statistika úložiště, tabulka logů
GET whoami 3.1	kdo jsem a s jakou rolí
GET POST admins, PATCH DELETE admins/:name, POST DELETE admins/:name/tokens[:id] 3.1	administrátoři, role, pojmenované tokeny (vlastník)
POST me/passkeys/options verify, POST auth/passkey/options verify signout 3.1	vlastní passkey administrátora; přihlášení do konzole passkey
GET audit/verify, POST audit/checkpoint 3.1	ověření hash řetězu a podpisů; podepsat hlavu teď
GET POST backups, POST db/integrity, POST db/optimize 3.1	zálohy, kontrola integrity, optimalizace ({vacuum:true} jen vlastník)
GET metrics, GET alerts, PUT alerts/rules/:id 3.1	metriky; pravidla, spuštěné alerty, historie; práh a zapnutí pravidla

```
# export auditu bezpečnostních událostí od úrovně warn do CSV
curl -H "Authorization: Bearer $ADMIN_API_TOKEN" \
  "https://chat.example.com/api/admin/audit/export?format=csv&category=security&minLevel=warn" -o audit.csv

# příkaz pro zařízení (doručí se hned, je-li připojené)
curl -X POST -H "Authorization: Bearer $ADMIN_API_TOKEN" -H "Content-Type: application/json" \
  -d '{"kind":"show-notification","deviceId":"dev-3f9c0a","payload":{"title":"Údržba","body":"Ve 22:00 restart"}}' \
  https://chat.example.com/api/admin/commands
```

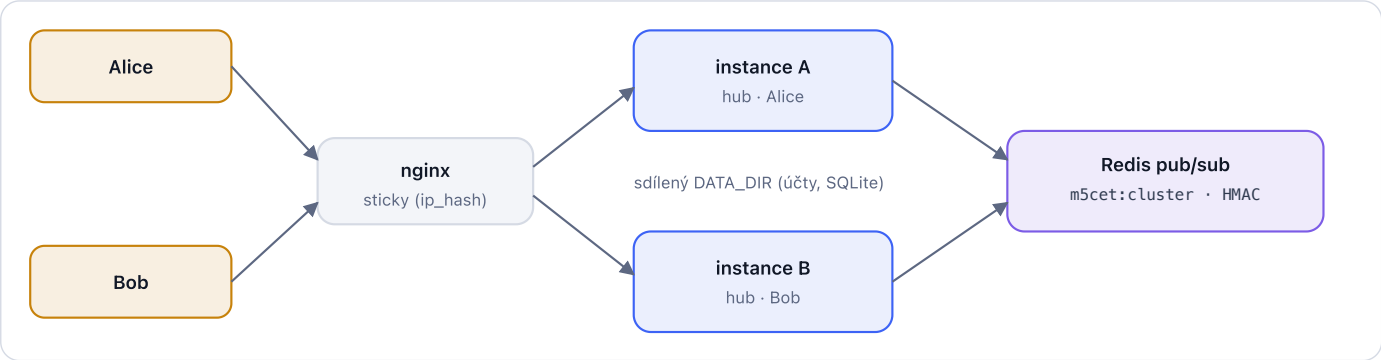
Admin služba (/admin/*)

Layout builder (/admin/layout), telefonie a SIP (/admin/telephony/* včetně sip/trunks), AI a řeč (/admin/plugins* , /admin/logs/stream).

Dřívější /admin/commands/* , /admin/clients , /admin/test/push , /admin/metrics a /admin/logs/recent dnes přeposílají na hlavní službu — ve 2.x čtly prázdnou kopii stavu v jiném procesu, takže příkazy nikdy nedorazily.

Více instancí 3.1

S `REDIS_URL` se místnost rozprostře přes všechny instance za load balancerem: lidé na různých instancích se vidí, spojí a pošlou si soubor přes relay.



Obr. 9 — Dvě instance, jedna místnost: členství, signály a relay jdou přes Redis.

ZPRÁVA	KDY	CO UDĚLÁ DRUHÁ INSTANCE
<code>join</code> / <code>leave</code> / <code>update</code>	člen vstoupí, odejde, přihlásí se	<code>peer-joined</code> / <code>peer-left</code> / <code>peer-updated</code> svým socketům; reference účtů si spočítá sama
<code>hello</code> → <code>state</code>	start instance, obnovené spojení s Redisem	pošle seznam svých členů po místnostech
<code>beat</code> / <code>bye</code>	každých 5 s / při zastavení	instance tichá 20 s (nebo s <code>bye</code>) zmizí i s členy
<code>signal</code>	SDP/ICE pro člena jinde	doručí adresátovi
<code>room</code> / <code>chunk</code>	rámec relay pro celou místnost	rozešle; binární kus binárně, starším klientům JSON
<code>to-sender</code>	<code>proxy-need</code> / <code>proxy-cancel</code> od příjemce	předá odesílateli přenosu, je-li u ní
<code>evict</code>	tentýž klient se vrátil (resume) na jinou instanci	starému socketu pošle <code>replaced</code>
<code>revoke</code> / <code>away</code>	odhlášení, smazání účtu / nepřítomnost	ukončí relace na svých socketech / srovná stav relay

Podpis zpráv

S `CLUSTER_SECRET` nese každá zpráva HMAC-SHA256; nepodepsané a padělané se zahodí (počítadlo `dropped`). Kdo smí do Redisu publikovat, bez tajemství místnosti neovládne. Bez tajemství server při startu varuje.

Sdílená data

Instance sdílejí `DATA_DIR` (jeden hostitel nebo sdílený svazek): SQLite úložiště a fronta jsou bezpečné pro více procesů (WAL), účty jsou JSON — instance si před každou operací načte, co zapsala jiná, a zapisuje hned.

Redis klient

Vlastní RESP klient bez závislostí: `redis://` i `rediss://` (TLS), `AUTH`, `SELECT`, reconnect s backoffem, znovu-přihlášení k odběru; zprávy odeslané při výpadku se po návratu odešlou (max. 1 000).

Omezení

Limity spojení a rámců jsou na instanci. Dvě instance měnící tentýž účet ve stejné milisekundě mohou jednu změnu ztratit — sticky sessions to prakticky vyloučí.

```
# nginx: dvě instance, sticky podle adresy klienta
upstream m5cet_app {
    ip_hash;
    server 127.0.0.1:5000;
    server 127.0.0.1:5001;
}
# každá instance
REDIS_URL=redis://:heslo@127.0.0.1:6379/0 CLUSTER_SECRET=$(openssl rand -base64 32) DATA_DIR=/var/lib/m5cet
```

Stav clusteru ukazuje konzole (Přehled › Zdraví) a metriky `m5cet_cluster_*`.

Nasazení

Proměnné prostředí

PROMĚNNÁ	VÝCHOZÍ	VÝZNAM
PORT, HOST	5000, 0.0.0.0	kde poslouchá hlavní služba
DATA_DIR	./m5cet	data: účty, úložiště, signaling.secret (systemd: StateDirectory)
TRUST_PROXY	loopback	kterým proxy věřit s X-Forwarded-For (limity, brána spojení, audit)
ALLOWED_ORIGINS 3.0	—	další originy směřjící otevřít /ws (čárkami); vlastní doména je povolena vždy
ADMIN_API_TOKEN	—	token konzole a admin API (bez něj vrací 503)
ENABLE_ADMIN, ADMIN_PORT, ADMIN_BIND	—, 5050, 127.0.0.1	admin služba
MAIN_URL 3.0	http://127.0.0.1:\$PORT	kam admin služba přeposílá /api/admin/*
AUDIT_COMMUNICATION 3.0	vypnuto	1 = audit komunikace od startu
LOG_HTTP 3.0	ve vývoji ano	1 = řádek na každý API požadavek i v produkci (bez těl)
PUSH_ENDPOINT_HOSTS 3.0	—	další povolené push služby (.example.com = i subdomény)
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_SUBJECT	—	Web Push
WEBAUTHN_RP_ID, WEBAUTHN_ORIGINS, PUBLIC_BASE_URL	—	doména passkeys a povolené originy; bez nich produkce varuje
STORAGE_DIR, STORAGE_MASTER_KEY, STORAGE_KEY_FILE	—	úložiště a jeho master klíč
ACCOUNTS_DIR, RELAY_RETENTION_DAYS	—, 30	účty; jak dlouho čeká položka fronty bez vlastního TTL
TURN_SERVER_URL, TURN_USERNAME, TURN_CREDENTIAL	—	TURN pro striktní NAT (statické přístupy — server varuje)
TURN_SECRET, TURN_TTL_SECONDS, STUN_URLS 3.1	—, 3600, Google STUN	krátkodobé TURN přístupy (coturn use-auth-secret); vlastní STUN servery
ADMIN_TOKENS 3.1	—	další administrátoři: jméno:role:token,... (role owner, operator, auditor)
METRICS_TOKEN 3.1	—	token pro /metrics (Prometheus); bez něj jen administrátor
ALERT_WEBHOOK_URL, ALERT_<PRAVIDLO> 3.1	—	kam posílat alerty; prahy pravidel (tabulka)
BACKUP_DIR, BACKUP_INTERVAL_HOURS, BACKUP_KEEP 3.1	—, 24, 7	plánované zálohy a kontroly integrity
REDIS_URL, CLUSTER_SECRET, CLUSTER_CHANNEL 3.1	—, —, m5cet:cluster	více instancí (cluster); tajemství pro podpis zpráv
WS_CONNECTS_PER_MINUTE, WS_CONNECTIONS_PER_CLIENT, WS_CONNECTIONS_TOTAL 3.1	30, 20, 5000	brána WebSocket spojení (na adresu klienta / celkem)
LOG_EVENTS	vypnuto	metadata událostí (/api/admin/events)
ENABLE_AI, ENABLE_SPEECH, ENABLE_TELEPHONY + klíče poskytovatelů	vypnuto	volitelné moduly

nginx

Konfigurace je v `deploy/nginx/m5cet.conf`. Pro 3.0 přibyl blok pro operátorské API (SSE potřebuje vypnutý buffering a dlouhý timeout):

```
location /api/admin/ {
    # allow 203.0.113.10; deny all; # doporučeno
    proxy_pass http://m5cet_app;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header Connection "";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 3600s;
}
```

Upgrade z 3.0 na 3.1

1. `git pull`, `npm ci` (nová závislost `hash-wasm`), `npm run build` (nebo `./update.sh`).
2. nginx: v hlavičce CSP `script-src 'self' 'wasm-unsafe-eval'` (hotová konfigurace v `deploy/nginx/m5cet.conf`); `nginx -t && systemctl reload nginx`. Bez toho Argon2id neběží a nikdo se nepřipojí.
3. Doporučeno: `TURN_SECRET` místo statických TURN přístupů, `BACKUP_DIR`, `METRICS_TOKEN`, `ALERT_WEBHOOK_URL`; jmenovití administrátoři místo sdíleného tokenu.
4. Restart `m5cet` a `m5cet-admin`. Migrace proběhnou samy: fronta dostane evidenci relayovaných zpráv a zapečetěná metadata (staré položky se dál přečtou), audit hash řetěz (staré záznamy „před řetězem“), účty nová pole.
5. **Všechna okna obnovit.** Klient 3.1 se připojuje pod slepým ID a jinými klíči — s klientem 3.0 se v místnosti nepotká. Zprávy čekající ve frontě z 3.0 se otevrou. Relace z 3.0 byly jen v paměti, takže se každý jednou znovu přihlásí passkey.

Upgrade z 2.x na 3.0

1. `git pull`, `npm ci`, `npm run build` (nebo `./update.sh`).
2. Do nginx přidat blok `/api/admin/`; `nginx -t && systemctl reload nginx`.
3. Běží-li hlavní služba jinde než na `127.0.0.1:$PORT`, nastavit admin službě `MAIN_URL`.
4. Restart `m5cet` a `m5cet-admin`. Při startu se staré schránky převedou do fronty (`queue.migrated` v auditu) a vznikne `signaling.secret`.
5. Otevřená okna nabídnou obnovení (nový build). Klient 2.x se s 3.0 nespojí — všichni obnoví.
6. Dřívější admin stránka ukládala token do `localStorage`; nová konzole ho při prvním otevření smaže. Zvažte výměnu `ADMIN_API_TOKEN`.

Ověření

```
npm run check          # TypeScript
npm test -- --no-file-parallelism # ~800 unit testů (vč. fuzz, upgrade z 3.0, cluster s Redisem)
npm run build
npm run test:e2e        # 48 testů: zprávy, soubory P2P i relay, šifrovaný hovor, účty a obnova, konzole a role...
npm run load-test -- --url ws://127.0.0.1:5000/ws --clients 500 # zátěž (zvednout WS.* limity na testovacím serveru)
curl -s https://chat.example.com/api/health # {"ok":true,"protocol":2,"version":"3.1.0","cluster":"local",...}
```

Bezpečnostní opatření serveru

HTTP

Helmet, CSP v produkci `script-src 'self' 'wasm-unsafe-eval'` (jen WebAssembly pro Argon2id, žádný `eval`) a `object-src 'none'`, workery jen z vlastního originu; `Cache-Control: no-store` všude; limity **před** parsey těl (veřejné API 100 / 15 min, trezor 300, úložiště 1 200, admin 30 odmítnutí / 15 min); JSON do 256 KB (trezor 8 MB, úložiště 12 MB); chyby 500 bez interních detailů.

Logy

Řádek požadavku = metoda, cesta, stav, doba, velikost. **Nikdy těla** — odpovědi nesou session tokeny a šifrované texty (2.x je logovala). Surové tělo se drží jen pro webhooky poskytovatelů kvůli podpisům.

WebSocket

Origin, brána spojení, maxPayload 256 KB, validace rámců, limity tříd, vyhazování opakovaných narušitelů, heartbeat, zpětný tlak, vlastnictví přenosů, rozesílání jen ověřených polí.

Identita a soukromí

Pseudonymy účtů v místnosti, obecná odmítnutí relay, IP v monitoru zkrácené na /24 (/48), místnosti jako hash, audit komunikace jen na vyžádání, zapečetěné detaily v databázi.

SSRF

Server POSTuje jen na push endpointy známých služeb (https, bez portu a přihlašovacích údajů); příkaz ke stažení souboru přijme jen https adresu a klient se uživatele zeptá.

Ukončení

SIGTERM/SIGINT: sockety se zavřou (klienti se vrátí přes resume), účty se zapíší, databáze se zavřou a klíče vynulují, pak konec (nejpozději za 8 s).

Návrhy a roadmapa

Návrhy z dokumentace 3.0 a jak dopadly ve 3.1; na konci to, co zbývá. Priorita **P1** = nejvyšší přínos pro bezpečnost či spolehlivost, náročnost S / M / L.

Stav návrhů z 3.0

OBLAST	NÁVRH	STAV	KDE
Kryptografie	Forward secrecy a odebrání člena	hotovo 3.1	klíče odesílatele s ratchetem, rotace, <i>Vyloučit z mých zpráv</i> — Šifrování v3. MLS zůstává v plánu pro velké skupiny.
	Slepá ID místností	hotovo 3.1	HKDF místo HMAC názvu, r3....
	Párové šifrování soukromých zpráv	hotovo 3.1	ECDH P-256 (WebCrypto nemá X25519 všude), podepsaný hello
	Identita svázaná s účtem	hotovo 3.1	Ed25519 z kořene účtu, certifikát zařízení, pin podle účtu
	Argon2id	hotovo 3.1	64 MiB / 3 průchody ve Web Workeru, CSP <code>'wasm-unsafe-eval'</code>
	Ověření QR kódem	hotovo 3.1	zobrazení i skenování (<code>BarcodeDetector</code>), potvrzení
Protokol	Více instancí	hotovo 3.1	Redis pub/sub, podepsané zprávy — Více instancí; fronta dál v SQLite (sdílený disk)
	Binární rámce	hotovo 3.1	vlastní binární formát kusů (jednodušší než CBOR) po DataChannelu i socketu; WebTransport zatím ne
	SFU s E2EE	částečně	rámce hovoru jsou šifrované end-to-end a připravené pro SFU; SFU samotné součástí není
	TURN s krátkodobými přístupy	hotovo 3.1	<code>TURN_SECRET</code> , obnova před vypršením
Fronta	Trvalá evidence účtenek	hotovo 3.1	tabulka <code>relay_ledger</code>
	Neutrální text probuzení	hotovo 3.1	push bez jména a místnosti
	Metadata fronty	hotovo 3.1	odesílatel a stav zapečetěné master klíčem, odesílatel jen jako hash (v globální DB — do DB uživatele se zapisovat nedá, když je zamčená)
Účty	Více passkeys a obnovovací klíč	hotovo 3.1	zapečetěný kořen pro každý passkey i kód
	Relace přežívající restart, seznam zařízení	hotovo 3.1	hashe v <code>sessions.json</code> , klouzavá platnost
	Zálohy a integrita	hotovo 3.1	online backup, manifest, quick_check, VACUUM
Administrace	Role, jmenné přístupy, přihlášení passkey	hotovo 3.1	vlastník / operátor / auditor; OIDC zatím ne
	Neměnný audit	hotovo 3.1	hash řetěz + podepsané body
	Prometheus, OpenTelemetry, alerty	částečně	<code>/metrics</code> a alerty s webhookem ano; trasování OpenTelemetry ne
Klient	Rozdělit <code>App.tsx</code> , error boundary	částečně	panely, ovládání hovorů, dialog a pomocné funkce v modulech, error boundary; hooky spojení a peerů zbývají
	Kryptografie ve Web Workeru	hotovo 3.1	KDF ve workeru, rámce hovoru ve workeru
	Rozdělení bundlu, dlouhé seznamy	hotovo 3.1	hlavní chunk 685 → 440 kB, React zvlášť; okno 200 zpráv + <code>content-visibility</code>
	Dokončit i18n	hotovo 3.1	hlášky chatu cs/en/de + test úplnosti

OBLAST	NÁVRH	STAV	KDE
Testy	Fuzzing a property testy	hotovo 3.1	vlastní seedovaný generátor (bez závislosti); našel chybu v RESP parseru
	Zátěžové testy	hotovo 3.1	<code>npm run load-test</code> (1 000 klientů lokálně: RTT p99 3 ms)
	E2E konzole v CI, test upgradu	hotovo 3.1	konzole s rolemi, obnova účtu, relay souborů, hovor; upgrade dat z 3.0

Co dál

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Relay a fronta pod klíči odesílatelů — předem rozeslané řetězy nebo prekeys (X3DH) pro nepřítomné	Zprávy pro nepřítomné a fronta odchozích jsou dnes pod klíčem místnosti: forward secrecy pro ně neplatí.	P1	L
Účty v SQLite místo JSON souborů	Bezpečné souběžné zápisy více instancí bez spoléhání na sticky sessions; transakce.	P2	M
SFU pro skupinové hovory (mediasoup / LiveKit) s již šifrovanými rámci	Mesh se nad 4–5 účastníků neškáluje; rámce jsou připravené.	P2	L
Hooky spojení a WebRTC peerů z <code>App.tsx</code> + jejich unit testy; stavový automat spojení	Zbytek logiky chatu je testovaný jen end-to-end.	P2	M
MLS (RFC 9420) pro velké místnosti	Párová distribuce klíčů odesílatelů roste s n^2 ; MLS s $n \log n$.	P3	L
Seznam zařízení účtu viditelný protistranám (podepsaný účtem, transparentní změny)	Upozornit, když účet získá nové zařízení, ne až když napíše.	P3	M
OIDC pro konzoli, OpenTelemetry trasování	Napojení na firemní identitu a tracing.	P3	M
Fronta v Postgresu , WebTransport	Cluster přes více hostitelů bez sdíleného disku; nižší latence relay.	P3	L
Post-kvantový hybrid (ML-KEM + ECDH) pro párové klíče	„Sklidit teď, rozšifrovat později“; až to WebCrypto nabídne nativně.	P3	M

Změny ve 3.1.0

Přidáno

Šifrování v3 (Argon2id ve workeru, slepá ID místností, klíče odesílatelů s ratchetem, párové klíče, identita účtu, QR bezpečnostní čísla), binární přenos kusů, relay souborů při selhání P2P s řízením rychlosti, E2EE rámců hovoru, krátkodobé TURN přístupy, cluster přes Redis, více passkeys a obnovovací kód, trvalé relace a seznam zařízení, role a přihlášení passkey do konzole, neměnný audit, zálohy, `/metrics` a alerty, error boundary, dělení bundlu, kompletní i18n, fuzz / zátěžové / upgrade testy.

Opraveno

Worker „background tick“ z `blob:` URL blokován produkční CSP (skrytá záložka neudržovala spojení); zvuk toho, kdo se k hovoru připojil druhý a nebyl iniciátorem, se nikdy nevyjednal; soubor bez P2P spojení nešel poslat vůbec; dva E2E testy sdílely port.

Změněno

Živé zprávy už nešifruje klíč místnosti; server zná jen slepé ID; metadata fronty zapečetěná; push probuzení neutrální; relace 12 h klouzavě (max. 7 dní) místo pevných 12 h v paměti; CSP povoluje WebAssembly.

Nekompatibilní

Klient 3.0 a 3.1 se v místnosti nepotkají (jiné ID i klíče) — po nasazení obnovit všechna okna. nginx potřebuje `'wasm-unsafe-eval'` v CSP.

Podrobný seznam: `CHANGELOG.md`. Tato dokumentace odpovídá kódu verze 3.1.0.

Změny ve 3.0.0

Přidáno

Protokol v2 (validace, limity, brána, Origin, resume, pseudonymy, `auth`, odvolání, heartbeat, zpětný tlak), šifrování v2 (klíče podle účelu, AAD, zapečetěná signalizace, podpisy, TOFU, ověřené soubory), SQLite fronta s lease a dead-letter, nová konzole s živým provozem, auditem, uživateli, frontou, úložištěm a systémem, dokumentace HTML + PDF.

Opraveno

XSS přes MIME souboru, DoS přes metadata souboru, převzetí peer ID, podvržené účtenky a položky schránky, logování session tokenů, SSRF přes push endpoint, token admina v localStorage, příkazy z admin služby, které nikdy nedorazily, vlastní zprávy po obnovení označené jako cizí, 18 oprav úložiště.

Změněno

Veřejný status úložiště bez statistik ostatních, metadata endpointy jen s tokenem, kód zapečetěné zprávy 12 znaků, transfer log bez jmen souborů, historie serveru bez passkey šifrovaná v prohlížeči.

Nekompatibilní

Klienti 2.x se s 3.0 nespojí (nezapečetěná signalizace) a nepřečtou obálky v2. Po nasazení obnovit všechna okna; přidat blok nginx `/api/admin/`.

Podrobný seznam: `CHANGELOG.md`.